

4.9 - RBAC, ABAC e ReBAC

Três formas de responder o que você pode fazer

RBAC: permissão por papel

O modelo que resolve a maioria dos casos

- **Usuário recebe papéis, papéis recebem permissões**
 - Admin, editor e leitor são exemplos clássicos
- **Simples de entender, auditar e implementar**
 - Resolve a maioria dos casos de produtos comuns
- **Quebra quando a regra depende de contexto**
 - Explosão de papéis sinaliza que RBAC não basta mais
- **Granularidade fina exige papéis demais para gerir**

Quatro tipos de usuário, uma regra difícil

- **Admin da plataforma, admin da transportadora, motorista, cliente**
 - RBAC simples resolve cerca de 80% dos casos

LUNALOG

A LunaLog tinha quatro tipos de usuário: administrador da plataforma, administrador da transportadora, motorista e cliente do e-commerce. RBAC simples resolvia uns 80% dos casos sem esforço. O problema vivia no restante: um administrador de transportadora podia gerenciar apenas os motoristas da própria transportadora, nunca de outra. Um motorista podia ver apenas as rotas do próprio dia. Essas regras dependiam de atributos e de relacionamentos, não só do papel, e RBAC puro simplesmente não dava conta. Ana percebeu que estava criando papéis cada vez mais específicos para tapar buracos que eram, na verdade, contextuais.

ABAC: permissão por atributo

- **Decisão usa atributos do usuário, recurso e contexto**
 - Departamento, horário, localização e dono entram na regra
- **Política avalia condições, não papéis fixos**
 - Mesma identidade tem acesso diferente por contexto
- **Expressivo o bastante para regras dinâmicas**
 - Custo: políticas mais difíceis de visualizar e testar
- **Ideal quando o acesso varia com a situação**

Política ABAC declarativa

PSEUDOCODE

```
permitir se:  
  usuario.transportadora_id == recurso.transportadora_id  
  e usuario.papel == 'admin_transportadora'
```

```
permitir se:  
  usuario.papel == 'motorista'  
  e recurso.tipo == 'rota'  
  e recurso.motorista_id == usuario.id  
  e recurso.data == hoje()
```

ReBAC: permissão por relacionamento

- **Acesso modelado como grafo de relacionamentos**
 - Usuário X é membro da organização Y
- **Permissão emerge do caminho entre os nós**
 - Inspirado no Zanzibar, motor de autorização do Google
- **Brilha em hierarquias e compartilhamento aninhado**
 - Pastas, equipes e documentos compartilhados em cascata
- **Custo: exige motor dedicado para consultar o grafo**

Como escolher entre os três

Comece com RBAC quando

- Permissões mapeiam bem para papéis
- Regras estáveis e pouco contextuais
- Time pequeno e produto inicial
- Auditoria simples é prioridade

Evolua para ABAC ou ReBAC quando

- Regras dependem de atributos dinâmicos
- Acesso varia por contexto e relacionamento
- Papéis explodem em número e exceção
- Multi-tenant com isolamento por dono



Você já sabe escolher o modelo de acesso certo. Mas imagine doze microserviços, cada um implementando autorização do próprio jeito: um com enum hardcoded, outro com if aninhado, outro com tabela no banco. Como garantir que a política seja a mesma, consistente, versionada e testável em todos eles?

Policy-as-code: OPA, Casbin e OpenFGA